Homage Software Architecture Review

Plain-English technical overview for the current Expo React Native and FastAPI local-first exploration app

Prepared July 5, 2026. This document explains how the software is organized today, what each major part does, and how the pieces communicate. Technical terms and acronyms are spelled out when introduced.

Project path reviewed: /Users/davidwilliamclark/projects/homage

Main app folder: app/. Main backend folder: backend/. Local database file: backend/data/homage.sqlite3.

1. Executive Summary

Homage is currently a local-first exploration app. "Local-first" means the app tries to keep useful identity, progress, artifacts, and generated media available on the user device or local backend first, instead of depending entirely on a remote internet service.

The software has two main halves:

- Mobile and web app: an Expo React Native application in app/. Expo is the development and build system. React Native is the user interface framework that lets the same code run on phones and web browsers.
- Backend service: a FastAPI application in backend/. FastAPI is a Python web server framework. It exposes Application Programming Interface endpoints, usually called API endpoints, that the app can call over Hypertext Transfer Protocol, usually called HTTP.

The most important architecture direction is to make the local SQLite database the main metadata and event store. SQLite is a small Structured Query Language database engine that stores data in one local file. The project already has backend/data/homage.sqlite3, and the backend already stores accounts, devices, players, quests, artifacts, pairing codes, and sync events there.

Files should still be used, but mostly for large exported or binary things: uploaded images, generated Portable Document Format files, LaTeX source files, song data, and audio files. The database should describe those files and track ownership, status, and history.

2. Glossary

Term	Meaning in this project
Expo	The development platform used to run, test, and build the React Native app. Expo Go is the phone app that loads the development build from your computer.
React Native	A JavaScript and TypeScript user interface framework for building phone apps. Homage also uses it for the web preview.
Expo Router	The navigation system that maps files in app/app/ to screens, such as /game, /admin, and /terminal.
FastAPI	A Python framework for building the backend web service. It receives requests from the app and returns structured responses.
API, Application Programming Interface	A defined way for one part of the software to ask another part for data or actions. Example: the app asks the backend to create a pairing code.
HTTP, Hypertext Transfer Protocol	The network protocol used when the app calls backend endpoints such as /healthz or /quests/generate.
JSON, JavaScript Object Notation	A text format for structured data. The app and backend send most non-file data as JSON.
SQLite, Structured Query Language database engine	The local database file used by the backend for accounts, devices, players, quests, artifacts, pairing codes, and sync events.
GPS, Global Positioning System	The phone location signal used for quest generation, map proximity, and sky alignment.
OSM, OpenStreetMap	The open map data source used to find nearby public features such as parks, trails, public art, historic places, and water.
QR code, Quick Response code	The scannable code that Expo displays so Expo Go can connect to the running development server.
SDK, Software Development Kit	A package of tools and libraries. Expo has an SDK version that expects matching package patch versions.

3. Current Folder Map

Path	Purpose
app/	The Expo React Native application. This includes screens, shared client libraries, sky logic, music logic, and local device storage helpers.
app/app/	Screen routes. For example, app/app/game.tsx becomes the Explore screen route /game.
app/lib/	Reusable app-side logic such as API calls, local artifact storage, melody storage, hunt progress, theme values, and environment quest fallbacks.
app/sky/	Astronomy, projection, star catalog, and live sky pose logic used by the Live Sky screen.
app/components/	Reusable user interface pieces, including multiplayer lobby, quest radar, photo review modal, and sky quest modal.
backend/	The FastAPI backend service. It exposes API endpoints and manages data stored on disk.
backend/local_database.py	SQLite database schema and database helper functions for local-first identity and indexed records.
backend/quests.py	Backend quest generation and safety filtering using OpenStreetMap data.
backend/server.py	Main backend server file. Defines request models, routes, file handling, room handling, document generation, song publishing, and local-first endpoints.
backend/data/homage.sqlite3	The current local SQLite database file.

4. Main Runtime Pieces

Piece	What it does	Current status
Expo Metro server	Runs the app during development and serves it to the web browser or Expo Go on a phone.	Running at http://localhost:8081 and exp://192.168.2.75:8081 when started with npx expo start --web -c.
Expo Go	Phone app that loads the development version from the Metro server.	Needs the phone to reach the Mac over the local network, unless tunnel mode is used.
FastAPI backend	Accepts API requests for health checks, uploads, quest generation, pairing, documents, songs, rooms, and artifact unlocks.	Implemented in backend/server.py.
SQLite database	Stores local-first metadata: accounts, devices, players, quests, artifacts, pairing codes, and sync events.	Schema version 1 in backend/local_database.py.
File storage	Stores large assets and generated files such as images, PDF files, LaTeX files, room JSON files, custom quest JSON files, and song JSON files.	Works now, but should gradually become secondary to database metadata.

5. App Screens and Responsibilities

Screen route	Purpose
/ or Home	Entry screen. Links to Explore, Admin Dashboard, Pair Device, Document Studio, Song Studio, Artifacts, and Terminal Help.
/game or Explore	Main exploration screen. Gets nearby quests, shows active quest panel or minimized widget, handles photos, melodies, sky completion, artifact unlocks, and multiplayer hunt behavior.
/admin	Local admin dashboard. Shows backend status, local database status, account counts, quest counts, document counts, song counts, and can create a pairing code.
/pair-device	Phone pairing screen. Claims a six-character code created by the desktop or admin screen and links the phone device to a local account.
/terminal	Allowlisted help and status console. It is not a real shell. It only runs built-in commands such as help, status, database, accounts, pairing, quests, artifacts, documents, songs, completed, sky, and clear.
/creator	Quest Studio. Lets a player create a community quest at a safe stationary location.
/document-studio	Document Studio. Creates poems, prose poems, or short stories, generates PDF and LaTeX output, and can publish the document as an artifact quest.
/song-studio	Song Studio. Creates chip-style songs with chord degrees, voicings, extensions, drums, melody, and tempo. Songs can be saved locally or dropped into the world.
/artifacts	Local artifact library. Shows saved photos, documents, and songs that the device has collected or created.
/melodies	Melody collection screen. Shows locally collected generated melodies.
/sky	Live Sky screen. Uses camera, motion sensors, location, astronomy calculations, and star catalog data to align the sky overlay.
/maze	Invisible Maze prototype. Uses map path generation and has known unrelated TypeScript errors at the time of review.

6. Identity and Pairing Model

The current model has three identity concepts. They are related, but they should stay separate because they answer different questions.

Concept	Question it answers	Stored where
Account	Which local household, base station, or owner does this device belong to?	Backend SQLite table local_accounts.
Device	Which physical phone, browser, or install is this?	Backend SQLite table devices and app-side profile storage.
Player	Who is playing or creating content inside the game?	Backend SQLite table players and app-side multiplayer profile storage.

Pairing flow today

1. The admin dashboard asks the backend to create a local account if needed and generate a six-character pairing code.
2. The backend writes that code to the pairing_codes table with an expiration time.
3. The phone opens Pair Device and sends the code plus its device identifier, device name, platform, player identifier, and player name.
4. The backend marks the code as claimed, registers the device, ensures the player exists, and records a sync event called device_paired.
5. The app saves the account identifier and account name into local AsyncStorage, which is persistent key-value storage on the device.

Recommended rule: pairing should link device to account. Player identity should remain a game profile that can later move across devices or accounts without rewriting the whole system.

7. Data Storage Layers

Storage layer	What lives there today	Recommended long-term role
Device AsyncStorage	Player profile, hunt progress, collected melody list, local artifact index, cached quest responses.	Good for device-local state and offline convenience. It should not be the only copy of shared or published metadata.
Device file system	Copied photos, downloaded PDF files, LaTeX source files, rendered song audio, generated melody audio.	Good for local files that should still work offline. Metadata should point to these files.
Backend SQLite database	Accounts, devices, players, quests, artifacts, pairing codes, sync events.	Should become the canonical local-first metadata and event store.
Backend JSON files	Rooms, custom quests, some document records, some song records.	Useful for quick prototyping, but should move toward SQLite metadata with files only for large assets.
Backend generated files	Uploaded images, processed files, PDF files, LaTeX files, song files.	Keep these as files. Store ownership, titles, quest links, access rules, and status in SQLite.

8. Backend Application Programming Interface

The backend exposes endpoints. An endpoint is a named web address path that accepts a request and returns a response. The app usually sends and receives JavaScript Object Notation data, except when uploading or downloading files.

Endpoint group	Examples	Purpose
Health and status	/, /healthz, /local/database	Check whether the backend is running and inspect the local database state.
Local-first identity	/local/accounts, /local/devices, /local/pairing-codes, /local/pairing-codes/{code}/claim	Create accounts, register devices, create pairing codes, and claim pairing codes.
Quest generation	/quests/generate, /quests/custom	Generate nearby environment quests and create or list custom community quests.
Artifacts and uploads	/upload, /files, /completed, /artifacts/unlock	Upload quest photos, list files, show completed records, and unlock protected documents or songs after a puzzle answer.
Documents	/documents, /documents/{document_id}.pdf, /documents/{document_id}.tex	Create written artifacts, generate PDF and LaTeX outputs, list documents, and serve protected files.
Songs	/songs, /songs/{song_id}	Create, validate, list, and retrieve published songs.
Multiplayer rooms	/multiplayer/rooms and room action endpoints	Create rooms, join rooms, track shared progress, and run hide-and-seek style flows.
Maze prototype	/maze/generate, /maze/test-osm	Prototype OpenStreetMap path generation for an invisible maze feature.

9. Quest Generation and Safety

Quest generation currently has both backend and client logic. The backend version in backend/quests.py is the authoritative version. It queries OpenStreetMap through Overpass, which is a public map query service, and chooses nearby public features. It then filters candidates for safety.

- Candidate places include public art, historic features, viewpoints, parks, trails, water features, bridges, and named pedestrian paths.
- Safety filtering rejects private, restricted, military, construction, abandoned, and unsafe access areas.
- Road buffering keeps quest destinations away from major and local motor roads, with an extra Global Positioning System error margin.
- The app asks for stationary confirmation for procedural or community destinations so the user does not chase a generated point unsafely.
- The client has a fallback generator in app/lib/environmentQuests.ts for degraded network conditions. It should stay more conservative than the backend.

Recommended rule: the backend owns safety policy. The client can cache and fall back, but it should reject outdated backend safety policy versions and clearly label cached or procedural results.

10. Artifacts, Melodies, Documents, and Songs

Feature	How it works today
Artifacts	An artifact is a saved memory from play: photo, document, or song. The app stores a local artifact index in AsyncStorage and copies files into the device file system. The backend also records artifact metadata in SQLite when uploads, documents, or songs are created.
Melodies	Melodies are generated locally from quest categories and seeds. The app stores collected melody metadata in AsyncStorage and audio files in the device file system.
Documents	Document Studio sends title, body, style, puzzle prompt, puzzle answer, and location safety data to the backend. The backend generates a PDF file and LaTeX source file, stores artifact metadata, and creates a quest that can unlock the document.
Songs	Song Studio creates chip-style compositions. The backend validates song structure, stores the song record, and creates a quest that can unlock the song. The app can render the song to audio locally for artifact playback.
Puzzle unlocks	The backend stores a hashed answer instead of the original answer. A hash is a one-way fingerprint. When the user answers correctly, the backend returns a tokenized file link.

11. Live Sky and Map Accuracy

The sky system is mostly device-side because it depends on real-time phone sensors. The app uses location, motion sensors, camera permission, astronomy calculations, and a star catalog to draw the current sky overlay.

- Location accuracy is checked before the sky experience is considered ready. The current target is roughly 35 meters or better with multiple location fixes.
- Compass and motion data are used to estimate phone heading, altitude angle, and roll.
- Astronomy calculations determine where the Sun and stars should appear for the current time and observer location.
- The sky overlay is used as a completion signal for quests. The backend does not need to calculate every sky frame; it only needs final completion metadata if shared progress is required.

Recommended rule: real-time sky pose stays on the device. Shared completion, artifact records, and quest state can go to the backend.

12. Terminal Help Console

The Terminal screen is intentionally not an unrestricted shell. A shell would let a user run arbitrary commands on the device or computer. Homage Terminal is an allowlisted help and admin console, meaning it only recognizes commands that are explicitly built into the app.

Command type	Examples	Meaning
Help and status	help, status, clear	Show available commands, summarize backend and local profile status, or clear output.
Local-first database	database, accounts, pairing	Show backend database counts, local account list, and this device pairing identity.
Game state	quests, completed, sky	Show custom quest counts, backend completed uploads, and an explanation of map and sky data sources.
Local collections	artifacts, melodies	Show local artifacts and local collected melodies saved on this device.
Published creations	documents, songs	Show documents and songs published by the current player.

Recommended rule: keep Terminal read-focused. If write commands are added later, they should be narrow, named commands such as reset local progress, never arbitrary command execution.

13. Current Technical Risks

- The backend has two persistence styles at once: SQLite metadata and JSON files. This is normal for a prototype, but it will become hard to synchronize unless SQLite becomes the main metadata store.
- The app has two API helper files: app/api.ts and app/lib/api.ts. This can cause confusion about which file is the source of truth for backend contracts.
- Quest generation safety logic exists on both backend and client. That creates a risk of the two versions drifting apart unless the backend remains authoritative and versioned.
- Expo package versions are behind the expected Expo Software Development Kit patch versions. The app currently bundles, but updates should be planned.
- TypeScript, the typed version of JavaScript used by the app, has known unrelated errors in app/maze.tsx and state/quests.ts. These should be isolated from architecture decisions but eventually cleaned up.
- The default Application Programming Interface base address in app.config.js currently points to an ngrok tunnel address. ngrok is a temporary public tunnel service. If that address changes, backend calls from Expo Go may fail until API_BASE is updated.

14. Recommended Architecture Going Forward

1. Use SQLite as the canonical local-first metadata store. Add tables or columns for custom quests, room state, document records, song records, artifact ownership, and sync status as those features mature.
2. Keep the backend file system for large assets only. Files should be named, owned, and indexed by database rows.
3. Create one app-side backend contract module. Prefer app/api.ts as the single place for endpoint types and fetch functions. Feature libraries can call it but should not redefine the same endpoint contracts.
4. Define a small domain model in plain terms: Account, Device, Player, Quest, Artifact, Completion, Room, PairingCode, SyncEvent, SkySignal.
5. Make every shared action record a sync event. Pairing, quest creation, artifact creation, upload completion, room action, and sky lock are all good event candidates.
6. Version safety policy and content schemas. The app already checks safety policy version for quests. Extend that idea to artifact records and room state as needed.
7. Keep Terminal as a status and recovery surface. It should explain what the system knows and help diagnose problems without becoming a hidden command shell.

15. Plain-English Data Flow Examples

Pairing a phone

1. Admin Dashboard asks the backend for a pairing code.
2. Backend creates or reuses a local account and stores a code in SQLite.
3. Phone enters the code on Pair Device.
4. Backend links the phone device to the account and records a device_paired sync event.
5. Phone saves the account information locally so it remembers the link.

Finding quests nearby

1. Explore gets the phone location.
2. The app asks the backend to generate nearby quests.
3. The backend queries OpenStreetMap, filters unsafe places, combines community quests, and returns up to three active quests.
4. If the backend or map service is unavailable, the app can use direct OpenStreetMap fallback, cached quests, or procedural stationary quests.

Creating a document artifact

1. Document Studio sends writing, style choices, puzzle data, player identity, and safe-location confirmation to the backend.
2. The backend creates a PDF and LaTeX source file.
3. The backend records the document as an artifact and creates a quest around it.
4. The app saves a local copy so the artifact can be reviewed later.

16. Immediate Next Steps

Priority	Work item	Why it matters
1	Write a short domain model document in the repository.	This gives every feature a shared vocabulary: account, device, player, quest, artifact, completion, room, and sync event.

Priority	Work item	Why it matters
2	Unify app API helpers into one source of truth.	Reduces future bugs where one screen calls an endpoint differently than another.
3	Move custom quest, document, song, and room metadata into SQLite tables.	Makes local-first sync, admin views, and recovery much easier.
4	Keep files for exported assets only.	PDF, LaTeX, audio, and image files are best stored as files, but the database should describe them.
5	Add a small backend status endpoint for app configuration.	Terminal and Admin Dashboard can show API base, database path, safety policy version, and storage health in one place.
6	Plan an Expo package patch update.	Keeps Expo Go compatibility cleaner and reduces warnings.

Bottom line: Homage already has the right shape for a local-first app. The next architecture move is not a rewrite. It is consolidation: one database for metadata and events, files for large assets, one client API contract, and a clear boundary between device-only experiences and backend-shared state.